

Game.py

Student worksheet - build the Texas Hold'em game loop

Create a file called game.py. Work through the steps in order. Each part adds one small piece to the final program. The final version should be able to run one full round of Texas Hold'em.

Do not worry if some classes, such as Player, Table, or ConsoleUI, are not yet implemented. Focus on understanding the overall game flow and how the classes interact with one another.

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of game.py, add the imports you will need.

```
from .cards      import Deck
from .evaluator  import HandEvaluator
from .player     import Player
from .table      import Table
from .ui         import ConsoleUI
```

Hint: The leading dot indicates that all imports come from the same package.

1. create the class TexasHoldemGame:

Note: All methods in steps 2 through 9 belong inside this class.

Inside `__init__`:

- accept players, small_blind, and big_blind as parameters
- raise ValueError if fewer than 2 players are provided
- create instances of Table, HandEvaluator, and ConsoleUI

```
class TexasHoldemGame:
    def __init__(self, players, small_blind=5, big_blind=10):
        ...
```

2. create the method play_hand(self) -> None:

- create a new Deck
- reset the table
- reset all players

- deal 2 cards to each player
- post the blinds
- deal the flop, turn, and river
- run betting rounds
- run the showdown

```
deck = Deck()
...

for player in self.players:
    ...
```

3. create the method `_post_blinds(self)`:

- make player 0 post the small blind
- make player 1 post the big blind
- add both amounts to the pot
- display a message in the format: "[name] posts [small]; [name] posts [big]"

```
small = self.players[0].bet(self.small_blind)
...
```

4. create the method `_show_human_cards(self) -> None`:

Loop over all players. If a player is human, display their hole cards.

5. create the method `_deal_community(self, deck, count, street)`:

- stop immediately if only one player remains
- draw the required number of community cards and add them to the table
- display the current street name

```
def _deal_community(self, deck: Deck, count: int, street: str) -> None:
    self.table.community_cards.extend(deck.draw(count))
    ...
```

6. create the method `_betting_round(self, street)`:

Main betting logic:

- skip players who have folded

- calculate the call amount for each active player
- ask each player for an action
- handle fold, call, and raise
- add chips to the pot
- reset `current_bet` at the end of the round

```
def _betting_round(self, street:str) -> None:
    ...
```

7. create the method `_bot_action(self, player, call_amount):`

Simplified bot logic:

- fold if the call amount exceeds half of the bot's remaining chips
- otherwise, call

```
def _bot_action(self, player: Player, call_amount: int) -> str:
    ...
```

8. create the method `_showdown(self):`

- evaluate all active (non-folded) players
- determine the winner
- split the pot if two or more players tie
- display the winning hand

```
rank = self.evaluator.best_rank(cards)
```

9. create the method `_only_one_player_left(self):`

Return True if only one non-folded player remains in the hand.

```
def _only_one_player(self) -> bool:
    ...
```

Common mistakes to check

- Forgetting to reset `current_bet` at the end of a betting round.
- Not adding bets to the pot.
- Evaluating players who have already folded.
- Forgetting to stop the round when only one player remains.